

Sistemas Operacionais e Computação em Nuvem

RELATÓRIO - MONITORAMENTO E ANÁLISE COM IA

Projeto NeuralVision

Nomes: Guilherme Muniz Gomes RA: 24026572

Maria Eduarda Cabral Foloni RA: 24026502

Lucas Moreira Godoy RA: 24026298

Flávia da Costa Rodrigues Faria RA: 20021548

1. Introdução:

Este relatório tem como objetivo desenvolver um sistema de monitoramento de recursos computacionais em ambiente de computação em nuvem, utilizando uma máquina virtual hospedada na plataforma Microsoft Azure. Além disso, foi aplicada Inteligência Artificial para análise dos dados coletados, permitindo identificar padrões de uso e realizar previsões de consumo de recursos.

2. O que foi utilizado:

- Plataforma: Microsoft Azure
- Tipo: Máquina Virtual (VM)
- Sistema Operacional: Ubuntu Server
- Acesso: SSH
- Linguagem: Python
- Bibliotecas:
 - psutil
 - pandas
 - matplotlib
 - scikit-learn

Obs: Foi utilizado um ambiente virtual (venv) para gerenciamento de dependências, garantindo isolamento das bibliotecas.

3. Implementação do monitoramento:

Foi desenvolvido um script em Python responsável por coletar dados de uso da máquina virtual, incluindo CPU, memória e disco. A coleta foi realizada em intervalos regulares de tempo, armazenando os dados em um arquivo CSV para análise posterior.

```

azureuser@NeuralVision:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
69 packages can be upgraded. Run 'apt list --upgradable' to see them.
azureuser@NeuralVision:~$ sudo apt install python3-pip -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
(venv) azureuser@NeuralVision:~$ pip install psutil pandas matplotlib scikit-learn
Collecting psutil
  Downloading psutil-7.2.2-cp36-abi3-manylinux2010_x86_64.manylinux_2_12_x86_64.manylinux_2_28_x86_64.whl.metadata (22 kB)
Collecting pandas
  Downloading pandas-3.0.2-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
 79.5/79.5 kB 2.7 MB/s eta 0:00:00
Collecting matplotlib
  Downloading matplotlib-3.10.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (52 kB)
 52.8/52.8 kB 3.3 MB/s eta 0:00:00
Collecting scikit-learn
  Downloading scikit_learn-1.8.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (11 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-2.4.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
azureuser@NeuralVision:~$ nano monitoramento.py

```

Código colocado dentro do nano **monitoramento.py**:

```

import psutil
import time
import pandas as pd
from datetime import datetime

dados = []

print("Coletando dados...")

for i in range(60): # coleta por 2 minutos (1 dado a cada 2s)
    cpu = psutil.cpu_percent()
    memoria = psutil.virtual_memory().percent
    disco = psutil.disk_usage('/').percent

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

```

```

dados.append([timestamp, cpu, memoria, disco])

print(f"{timestamp} | CPU: {cpu}% | Memória: {memoria}% | Disco: {disco}%")

time.sleep(2)

df = pd.DataFrame(dados, columns=["timestamp", "cpu", "memoria", "disco"])
df.to_csv("dados_monitoramento.csv", index=False)

print("Coleta finalizada!")

```

```

(venv) azureuser@NeuralVision:~$ python3 monitoramento.py
Coletando dados...
2026-05-03 23:09:43 | CPU: 66.7% | Memória: 4.1% | Disco: 9.7%
2026-05-03 23:09:45 | CPU: 0.3% | Memória: 4.1% | Disco: 9.7%
2026-05-03 23:09:47 | CPU: 0.0% | Memória: 4.1% | Disco: 9.7%
2026-05-03 23:09:49 | CPU: 0.0% | Memória: 4.1% | Disco: 9.7%
2026-05-03 23:11:33 | CPU: 0.0% | Memória: 4.1% | Disco: 9.7%
2026-05-03 23:11:35 | CPU: 0.0% | Memória: 4.1% | Disco: 9.7%
2026-05-03 23:11:37 | CPU: 0.0% | Memória: 4.1% | Disco: 9.7%
2026-05-03 23:11:39 | CPU: 0.0% | Memória: 4.1% | Disco: 9.7%
2026-05-03 23:11:41 | CPU: 0.5% | Memória: 4.1% | Disco: 9.7%
Coleta finalizada!

```

4. Armazenamento dos dados

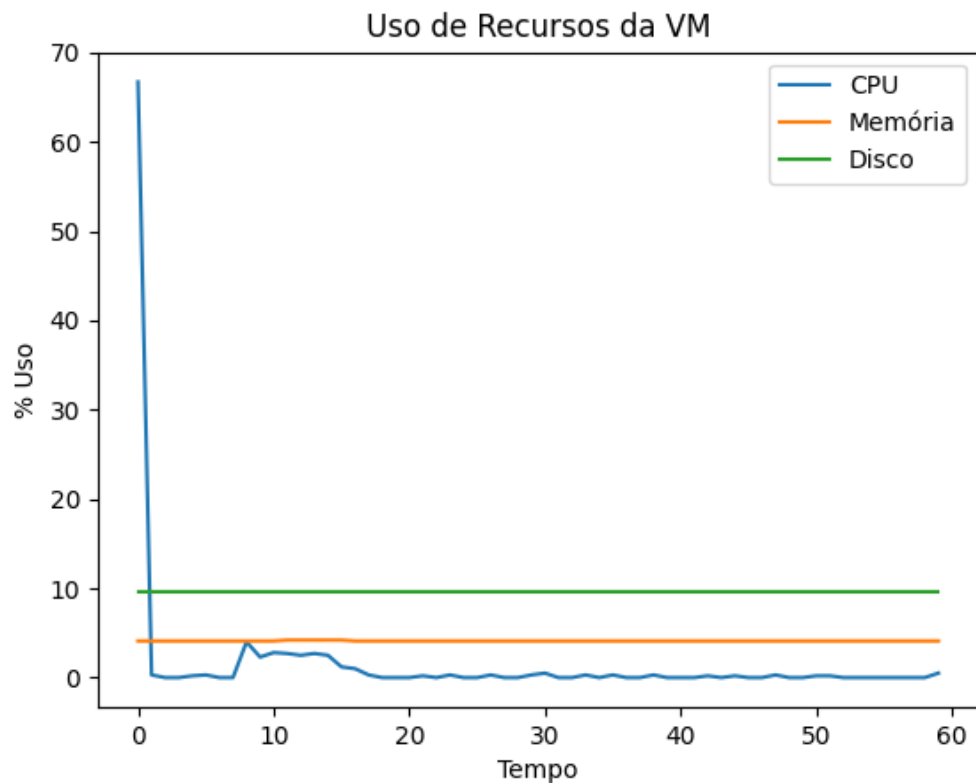
Os dados coletados foram armazenados em um arquivo CSV, contendo informações estruturadas sobre o uso de CPU, memória e disco ao longo do tempo.

[dados_monitoramento.csv.xlsx](#)

	A	B	C	D	E	F
1	timestamp	cpu	memoria	disco		
2	03/05/2026 23:09	667	41	97		
3	03/05/2026 23:09	3	41	97		
4	03/05/2026 23:09	0	41	97		
5	03/05/2026 23:09	0	41	97		
6	03/05/2026 23:09	2	41	97		
7	03/05/2026 23:09	3	41	97		
8	03/05/2026 23:09	0	41	97		
9	03/05/2026 23:09	0	41	97		
0	03/05/2026 23:09	40	41	97		
1	03/05/2026 23:10	23	41	97		
2	03/05/2026 23:10	28	41	97		
3	03/05/2026 23:10	27	42	97		
4	03/05/2026 23:10	25	42	97		
5	03/05/2026 23:10	27	42	97		
6	03/05/2026 23:10	25	42	97		
7	03/05/2026 23:10	12	42	97		
8	03/05/2026 23:10	10	41	97		
9	03/05/2026 23:10	3	41	97		
0	03/05/2026 23:10	0	41	97		
1	03/05/2026 23:10	0	41	97		
2	03/05/2026 23:10	0	41	97		

5. Análise dos dados

Os dados coletados foram analisados utilizando gráficos, permitindo visualizar o comportamento dos recursos da máquina virtual ao longo do tempo.



6. Aplicação de inteligência artificial

Foi aplicado o algoritmo KMeans para agrupar os dados em diferentes categorias de uso (baixo, médio e alto consumo). Essa abordagem permitiu identificar padrões no comportamento da máquina virtual.

```
(venv) azureuser@NeuralVision:~$ nano analise_ia.py
(venv) azureuser@NeuralVision:~$ python3 analise_ia.py
```

Código da **analise_ia.py**:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.linear_model import LinearRegression

# carregar dados
df = pd.read_csv("dados_monitoramento.csv")

# =====
# GRÁFICOS
# =====
```

```

plt.figure()
plt.plot(df["cpu"], label="CPU")
plt.plot(df["memoria"], label="Memória")
plt.plot(df["disco"], label="Disco")
plt.legend()
plt.title("Uso de Recursos da VM")
plt.xlabel("Tempo")
plt.ylabel("% Uso")
plt.show()

# =====
# IA 1 — Clusterização (KMeans)
# =====
kmeans = KMeans(n_clusters=3)
df["cluster"] = kmeans.fit_predict(df[["cpu", "memoria", "disco"]])

print("\nClusters encontrados:")
print(df[["cpu", "memoria", "disco", "cluster"]].head())

# =====
# IA 2 — Previsão (Regressão)
# =====
df["tempo"] = range(len(df))

X = df[["tempo"]]
y = df["cpu"]

modelo = LinearRegression()
modelo.fit(X, y)

previsao = modelo.predict(X)

plt.figure()
plt.plot(df["tempo"], y, label="CPU Real")
plt.plot(df["tempo"], previsao, label="Previsão CPU")
plt.legend()
plt.title("Previsão de Uso de CPU")
plt.xlabel("Tempo")
plt.ylabel("% CPU")
plt.show()

```

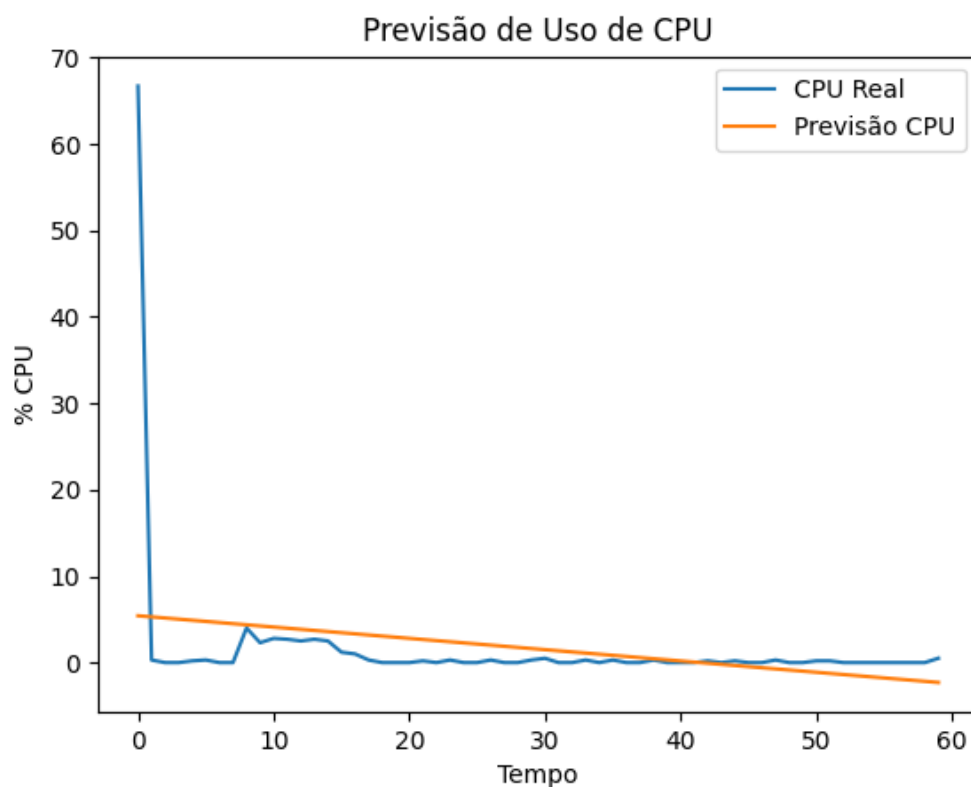
```
(venv) azureuser@NeuralVision:~$ python3 analise_ia.py
```

Clusters encontrados:

	cpu	memoria	disco	cluster
0	66.7	4.1	9.7	1
1	0.3	4.1	9.7	0
2	0.0	4.1	9.7	0
3	0.0	4.1	9.7	0
4	0.2	4.1	9.7	0

6.1 Regressão Linear (Previsão)

Também foi utilizado um modelo de regressão linear para prever o comportamento futuro do uso de CPU com base nos dados coletados.



7. Análise dos resultados

A análise dos dados demonstrou que o uso de CPU apresentou variações ao longo do tempo, possivelmente relacionadas à execução de processos na máquina. A clusterização permitiu identificar diferentes níveis de carga, enquanto a regressão indicou uma tendência de comportamento relativamente estável, com pequenas oscilações. Para resolver o problema de instalação de bibliotecas, foi utilizado um

ambiente virtual Python (venv). Para transferência de arquivos, foi utilizado o comando SCP com o IP correto da máquina virtual.

8. Conclusão

O projeto demonstrou a importância da integração entre computação em nuvem e Inteligência Artificial, permitindo não apenas monitorar recursos, mas também analisar e prever comportamentos do sistema. Essa abordagem pode ser aplicada em cenários reais para otimização de desempenho e tomada de decisão.

9. Relação com o projeto NeuralVision

O monitoramento de recursos pode ser utilizado para garantir o desempenho do sistema de visão computacional do projeto NeuralVision, evitando sobrecarga da máquina e garantindo maior eficiência no processamento de imagens/alimentos que será capturado pela câmera.